



Como funciona um site

Front, back, request e response, JavaScript, DOM, eventos, terminal e siglas

Como este material está organizado

Um resumo para explicar, passo a passo, como um site funciona: quem faz o quê, como cliente e servidor conversam, o que é o JavaScript e o DOM, para que serve o terminal e o que significam as siglas que aparecem toda hora. Cada parte tem um desenho para facilitar a explicação.

- 1. O que é um site
- 2. Front-end e back-end
- 3. Request e response
- 4. HTML, CSS e JS
- 5. JavaScript por dentro
- 6. O DOM
- 7. Eventos
- 8. Terminal e console
- 9. Siglas e palavras importantes
- 10. O caminho de um clique
- Frases para lembrar

Como usar

Leiam um bloco, olhem o desenho e tentem explicar com as próprias palavras antes de passar para o próximo.

1. O que é um site

Um site é um conjunto de arquivos (HTML, CSS, JS, imagens, sons) guardados em um computador ligado na internet. Esse computador se chama **servidor**.

Quando você digita um endereço no navegador, ele pede esses arquivos ao servidor, recebe e monta a página na tela. O seu computador ou celular, que pede, é o **cliente**.

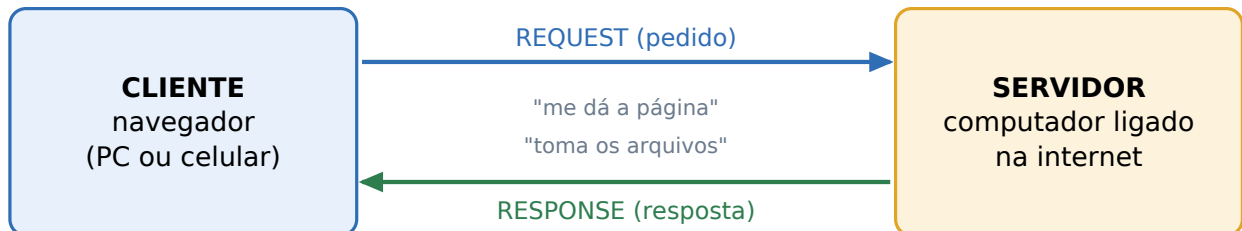


Figura 1 – O cliente pede, o servidor responde.

2. Front-end e back-end

Front-end é tudo o que a pessoa vê e mexe: cores, botões, textos, animações. Roda no navegador de quem acessa e é feito com HTML, CSS e JS.

Back-end é o que acontece escondido no servidor: guardar contas, conferir senha, salvar mensagens, contar votos. Pode ser feito em linguagens como Python, usando um **banco de dados** para guardar tudo em tabelas. Se o servidor desligar, os dados continuam lá.

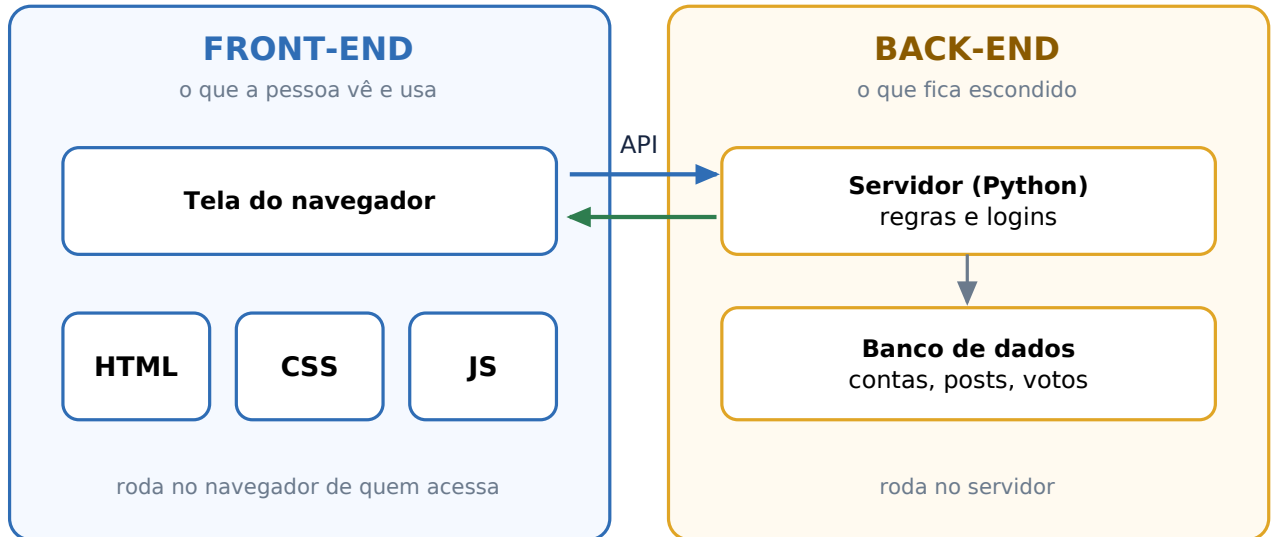


Figura 2 – Front e back conversam pela API.

Por que separar?

Tudo o que está no front pode ser visto e mexido por qualquer pessoa (o F12 mostra tudo). Regras importantes e dados secretos ficam no back. Exemplo: um texto que só deve aparecer numa certa data precisa ser liberado pelo servidor. Se estivesse só no front, qualquer um leria antes da hora.

Para lembrar

Front é a vitrine. Back é a cozinha e o estoque.

3. Request e response

Request (requisição) é o pedido que o cliente faz. **Response** (resposta) é o que o servidor devolve. Sempre começa pelo cliente, e o servidor só responde. Ele não chama o cliente por conta própria.

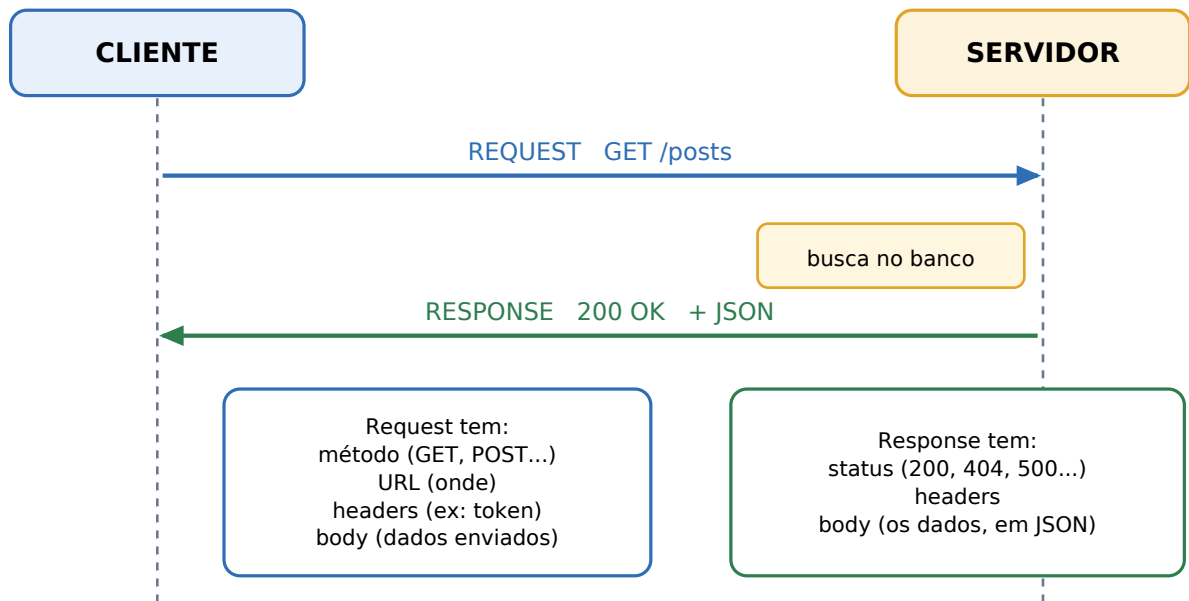


Figura 3 – Um pedido e uma resposta.

Métodos mais comuns

Método	Para que serve	Exemplo
GET	buscar ou ler	ver os posts
POST	criar algo novo	fazer login, criar um post
PUT	trocar uma coisa inteira	editar um post
DELETE	apagar	apagar um post

Status codes que vale decorar

Código	Significado
200	OK, deu certo
201	criado com sucesso
400	pedido errado (faltou um dado, por exemplo)
401	não está logado
403	logado, mas sem permissão
404	não encontrado
500	erro no servidor

Regra rápida

Começou com **2**: deu certo. Começou com **4**: o erro foi de quem pediu. Começou com **5**: o erro foi do servidor.

JSON

É o formato de texto usado para trocar dados. Parece um objeto do JS:

JSON

```
{ "id": 7, "autor": "luna", "votos": 12 }
```

Pedir algo ao servidor leva tempo, então o JS precisa esperar a resposta sem travar a página. O `fetch` faz o pedido e usa `async` e `await` para esperar.

4. HTML, CSS e JS

São as três partes do front. Cada uma cuida de uma coisa.



Figura 4 – Estrutura, aparência e comportamento.

Parte	Cuida de	Exemplo
HTML	estrutura: o que existe na página	título, botão, parágrafo, imagem, lista
CSS	aparência: como cada coisa parece	cor, tamanho, posição, fonte, espaço
JS	comportamento: o que acontece quando alguém faz algo	clicou, digitou, o tempo passou

Dica de fala

HTML é o esqueleto, CSS é a roupa e JS é o músculo. Os três se encontram pelo `id` e pela `class` dos elementos: o nome tem que ser igual nos três arquivos.

5. JavaScript por dentro

JavaScript (JS) é uma linguagem de programação. Nasceu para rodar no navegador e deixar a página viva. Hoje também roda fora dele (Node.js), mas aqui falamos do JS do navegador.

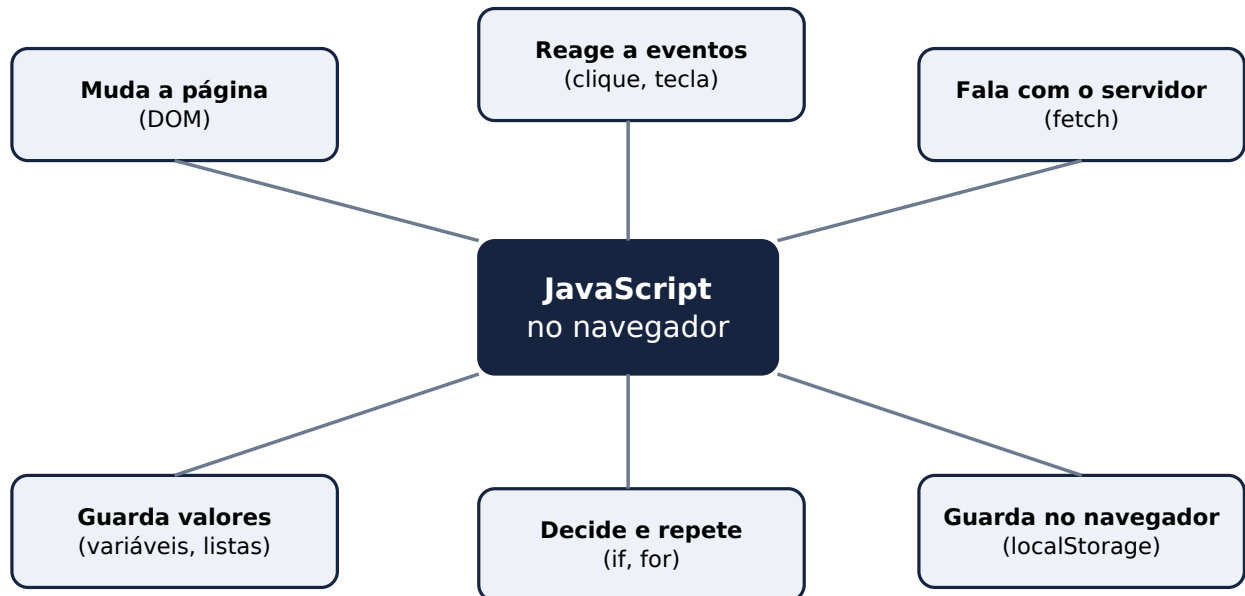


Figura 5 – O que o JS faz no navegador.

Ideias básicas da linguagem

Ideia	O que é	Exemplo
Variável	caixa com nome que guarda um valor. <code>const</code> não muda, <code>let</code> pode mudar	<code>const nome = "Ana";</code>
Tipos	string (texto), number, boolean (true/false), array, object, null	<code>"oi"</code> 42 true
Condição	decide entre caminhos	<code>if (idade >= 18) { ... } else { ... }</code>
Repetição	faz algo várias vezes	<code>for (let i = 0; i < 5; i++) { ... }</code>
Função	bloco de código com nome, chamado quando quiser	<code>function dobrar(n) { return n * 2; }</code>
Array	lista de valores	<code>[10, 20, 30]</code>
Objeto	agrupa dados com nome	<code>{ autor: "luna", votos: 12 }</code>
<code>console.log</code>	mostra algo no console, ótimo para investigar	<code>console.log("oi")</code>

JS

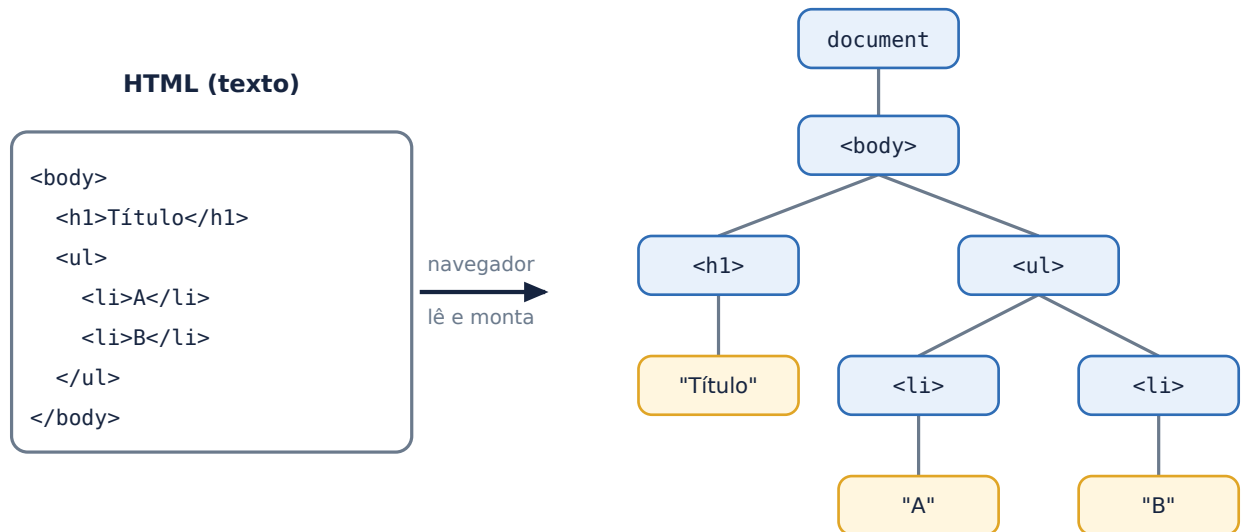
```
const nome = "Ana";  
let pontos = 0;  
pontos = pontos + 10;  
  
function dobrar(n) {  
  return n * 2;  
}  
  
if (pontos >= 10) {  
  console.log("Passou, " + nome);  
}  
console.log(dobrar(pontos));
```

SAÍDA NO CONSOLE

```
Passou, Ana  
20
```

6. O DOM

DOM significa Document Object Model. Quando o navegador recebe o HTML, ele transforma o texto em uma **árvore** de objetos na memória. Cada tag vira um nó. O JS não mexe no arquivo HTML: ele mexe nessa árvore, e o navegador redesenha a tela com o que mudou.



DOM: a árvore na memória. O JS mexe aqui e a tela muda.

Figura 6 - Do HTML à árvore do DOM.

Palavra	Significado
pai	quem está acima (o <code>ul</code> é pai dos <code>li</code>)
filho	quem está dentro (os <code>li</code> são filhos do <code>ul</code>)
irmãos	os que têm o mesmo pai
nó	cada ponto da árvore
elemento	um nó que é uma tag

O que o JS faz com o DOM

Verbo	Comandos	Observação
Achar	<code>document.getElementById("x")</code> <code>document.querySelector(".x")</code> <code>document.querySelectorAll(".x")</code>	<code>querySelector</code> devolve o primeiro, <code>querySelectorAll</code> devolve todos
Ler e mudar conteúdo	<code>elemento.textContent = "novo"</code> <code>elemento.innerHTML = "oi"</code>	<code>innerHTML</code> aceita tags; com texto digitado por usuário é perigoso (XSS). Prefira <code>textContent</code>
Mudar visual	<code>classList.add("x")</code> <code>classList.remove("x")</code> <code>classList.toggle("x")</code>	melhor trocar classes do que mexer em <code>style</code> direto
Criar e remover	<code>document.createElement("li")</code> <code>lista.append(li)</code> <code>li.remove()</code>	criar não basta: tem que colocar na página com <code>append</code>

JS

```
const lista = document.getElementById("lista");

const li = document.createElement("li");
li.textContent = "C";
lista.append(li);

lista.classList.add("destaque");
```

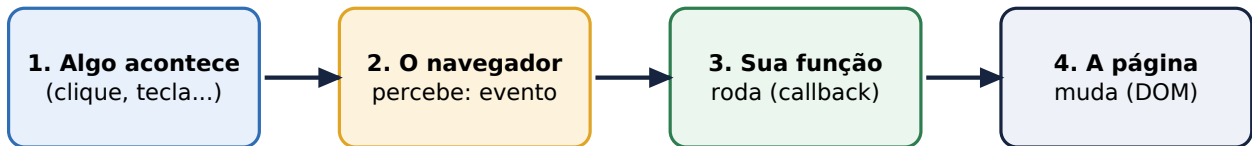
Dica de fala

O DOM é como o HTML fica depois que o navegador lê. O JS não mexe no arquivo, mexe na cópia viva que está na memória.

7. Eventos

Event (evento) é qualquer coisa que acontece na página e que o navegador avisa: clique, tecla apertada, formulário enviado, mouse passando por cima, tempo passando, página carregando.

O padrão é sempre o mesmo: **quando ESTE elemento receber ESTE evento, execute ESTA função**. A função que roda se chama **callback** (ou manipulador). O navegador chama ela, você não chama.



`addEventListener("click", funcao) = "quando clicar, rode esta função"`

Figura 7 – O caminho de um evento.

JS

```
botao.addEventListener("click", function (evento) {  
  console.log("clique");  
  console.log(evento.target);  
});
```

O parâmetro `evento` (também escrito `e` ou `event`) é um objeto com informações sobre o que aconteceu.

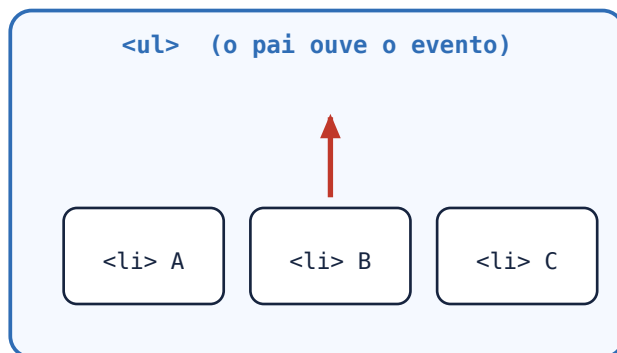
Propriedade ou método	O que faz
<code>evento.target</code>	o elemento exato onde aconteceu
<code>evento.key</code>	qual tecla foi apertada (eventos de teclado)
<code>evento.preventDefault()</code>	cancela o comportamento padrão, por exemplo impedir um formulário de recarregar a página

Eventos mais usados

Evento	Quando acontece
<code>click</code>	clique
<code>submit</code>	formulário enviado
<code>input</code>	alguém digitou em um campo
<code>keydown</code>	tecla apertada (ex: Esc)
<code>hashchange</code>	o endereço mudou depois do #
<code>load</code>	a página terminou de carregar

Delegação de eventos (event delegation)

Em vez de colocar um ouvinte em cada item de uma lista, coloca um só no **pai**. Como o evento “sobe” dos filhos até o pai (**bubbling**), o pai descobre quem foi clicado com `evento.target`. Serve muito quando os elementos são criados depois pelo JS.



Você clica no B.

O evento "sobe" até o .

No uma função só descobre quem foi clicado com:

`evento.target`

Figura 8 – O evento sobe do filho para o pai.

8. Terminal e console

O **terminal** é uma janela onde você digita comandos de texto para o computador em vez de clicar. O **console do navegador** (F12, aba Console) é outra coisa: é onde o JavaScript da página aparece.

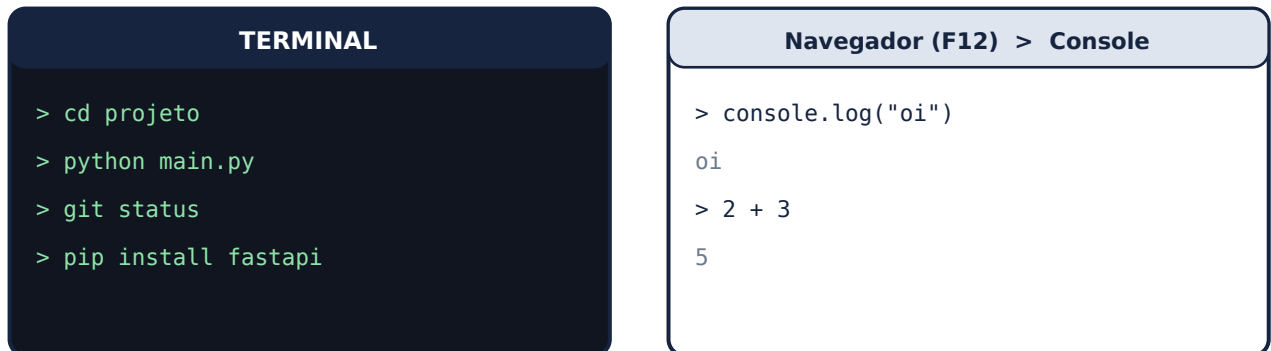


Figura 9 – Terminal e console não são a mesma coisa.

Para que serve o terminal

- rodar programas (`python main.py`)
- iniciar o servidor (`uvicorn main:app --reload`)
- instalar bibliotecas (`pip install fastapi`)
- usar o Git (`git add`, `git commit`, `git push`)
- navegar pelas pastas

Comando	O que faz
<code>cd pasta</code>	entra em uma pasta
<code>cd ..</code>	volta uma pasta
<code>ls</code> (ou <code>dir</code>)	lista o que tem na pasta
<code>mkdir nome</code>	cria uma pasta
<code>pip install x</code>	instala uma biblioteca Python
<code>git status</code>	mostra o que mudou no projeto

As abas do F12

Aba	Para que serve
Elements	mostra o DOM ao vivo e o CSS aplicado
Console	mensagens do <code>console.log</code> e erros do JS em vermelho; dá para digitar JS
Network	todos os requests e responses; ótima para entender o que o site pede ao servidor
Application	mostra o <code>localStorage</code>

Resumo

Terminal é a conversa com o computador. Console é a conversa com o JS da página.

9. Siglas e palavras importantes

Termo	Significado
HTML	HyperText Markup Language. Descreve a estrutura da página
CSS	Cascading Style Sheets. Folha de estilo, a aparência
JS	JavaScript. O comportamento
DOM	Document Object Model. A árvore da página que o JS mexe
URL	endereço de um recurso (https://site.com/pagina)
HTTP / HTTPS	o “idioma” que cliente e servidor usam. HTTPS é a versão com criptografia (o cadeadinho)
API	Application Programming Interface. As “portas” que o back oferece ao front, cada uma com URL e método
REST	jeito comum de organizar uma API com URLs e os métodos GET, POST, PUT, DELETE
Endpoint	uma das URLs da API (ex: /login)
JSON	formato de texto para trocar dados
Request / Response	pedido / resposta
Token	código que o servidor dá depois do login; o front o manda nos próximos pedidos para provar quem é
Event	algo que aconteceu na página
Callback	função chamada quando algo acontece
Listener	ouvinte: o que fica “escutando” um evento
Bubbling	o evento sobe dos filhos para os pais
Assíncrono	que não trava o resto enquanto espera (fetch, setTimeout)
Promise, async, await	jeito de lidar com coisas assíncronas
localStorage	pequeno armazenamento do navegador, só texto, fica salvo mesmo fechando a aba
XSS	ataque em que alguém coloca código malicioso em um texto que o site depois mostra como HTML
Hash	parte do endereço depois do # (pagina.html#galeria), usada para trocar de “página” sem recarregar
SPA	Single Page Application: site de um HTML só que troca o conteúdo com JS
Framework / biblioteca	código pronto de terceiros
Git / GitHub	Git controla versões do código; GitHub guarda o projeto na internet e deixa várias pessoas trabalharem juntas
Debug	procurar e consertar erros no código
Deploy	colocar o site no ar
Cache	cópia guardada para não pedir de novo

10. O caminho de um clique

Exemplo: uma pessoa clica em “Votar” em um post. Veja tudo o que acontece, do clique até a tela mudar.

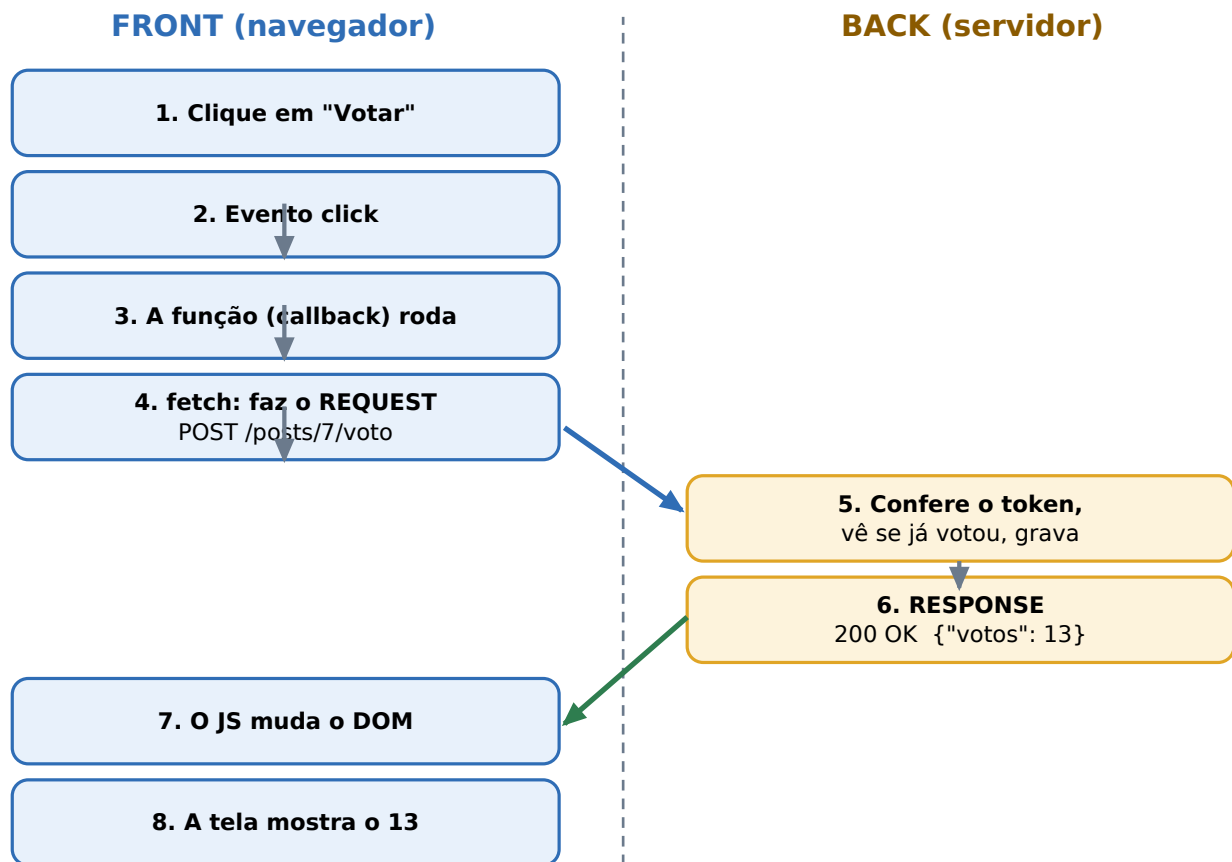


Figura 10 – Front e back trabalhando juntos.

JS

```
botao.addEventListener("click", async function () {  
  const resposta = await fetch("/posts/7/voto", { method: "POST" });  
  const dados = await resposta.json();  
  carta.querySelector(".votos").textContent = dados.votos;  
});
```

Nessa ordem: o evento chama a função, o `fetch` faz o request, o servidor responde com JSON e o JS usa o DOM para mostrar o novo valor.

Frases para lembrar

Front roda no navegador. Back roda no servidor.

O cliente sempre pede, o servidor só responde.

O JS não mexe no HTML, mexe no DOM.

Evento é o aviso, callback é o que a gente faz com ele.

Console é o terminal do JS. Terminal é o do computador.

O que está no front, qualquer um vê. Segredo fica no back.

Texto de usuário vai com `textContent`, não com `innerHTML`.